# Cidades ESG Inteligentes

Zeladoria Cidadã

Pipeline CI/CD, containerização e deploy em staging e produção


**Integrantes**

Mateus Bhering Beltrão Santos - RM564760

Ariel Alves Amaral - RM564563

Wesley Santos de França - RM563666

Ronaldo dos Santos Silva - RM561414

Rodrigo Kumamoto Rêgo - RM566049


Fase: Navegando pelo mundo DevOps

Repositório: https://github.com/mateusbhering/ATIVIDADE-DESAFIO-DEVOPS

# 1. Visão geral: Zeladoria Cidadã

A aplicação é uma API REST em Java 21 com Spring Boot 3.3 e PostgreSQL para o morador relatar problemas urbanos (buraco na via, poste apagado, descarte irregular de lixo, vazamento de água) e acompanhar o atendimento por um número de protocolo. Cada tipo de ocorrência tem uma secretaria responsável e um prazo; um painel público mostra, por secretaria, quantas ocorrências foram resolvidas no prazo e o tempo médio de resolução.

| Pilar ESG | Como aparece no projeto |
|---|---|
| Ambiental | Ocorrências de descarte irregular, poda de árvores e vazamento de água |
| Social | Canal direto do cidadão com a prefeitura e prazos iguais para todos os bairros |
| Governança | Fluxo auditável com datas e observação obrigatória, e painel público de transparência |

## Regras de negócio

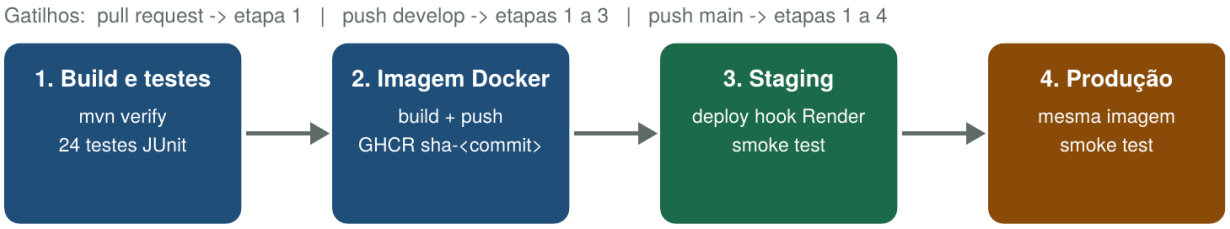
- Fluxo de status: ABERTA → EM_ATENDIMENTO → RESOLVIDA, com CANCELADA possível nas duas primeiras etapas. Pular etapa ou reabrir uma ocorrência encerrada retorna 409 Conflict.
- Prazos por tipo: 24 h (vazamento), 48 h (iluminação), 72 h (buraco e descarte), 96 h (poda) e 120 h (calçada).
- Ocorrência pendente com prazo vencido é marcada como atrasada e aparece em /api/ocorrencias/atrasadas.
- Resolver ou cancelar exige observação, que fica registrada para a prestação de contas.

| Ambiente | Onde roda | Endereço |
|---|---|---|
| Local | Docker Compose (app + PostgreSQL) | http://localhost:8080 |
| Staging | Render Web Service (branches develop e main) | https://cidades-esg-staging-cxn4.onrender.com |
| Produção | Render Web Service (somente main) | https://cidades-esg-prod.onrender.com |

O endpoint **/api/info** informa o ambiente e o commit em execução, e o **/actuator/health** informa a saúde da aplicação; ambos são usados pelo pipeline para confirmar cada deploy.

# 2. Pipeline CI/CD

**Ferramenta:** GitHub Actions (arquivo .github/workflows/ci-cd.yml). Registro de imagens: GitHub Container Registry (GHCR). Hospedagem: Render, com um Web Service por ambiente e um PostgreSQL gerenciado.




*Se qualquer etapa falhar, as seguintes não executam. A imagem é construída uma vez e promovida.*

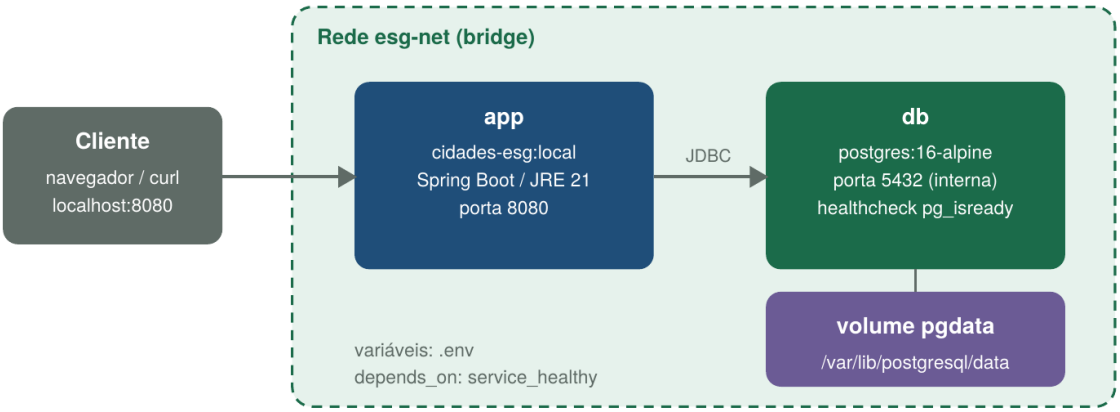
| Etapa | Quando | O que faz |
|---|---|---|
| 1. Build e testes | PR, develop, main | Java 21 + cache Maven; mvn verify compila e roda 24 testes (13 unitários das regras de negócio e 11 de integração da API com MockMvc e H2). Publica relatórios e o .jar como artefatos e o resumo dos testes. |
| 2. Imagem Docker | develop, main | Buildx constrói a imagem pelo Dockerfile e publica no GHCR com as tags sha-<commit> e o nome da branch. O commit é gravado na imagem (APP_VERSION). |
| 3. Deploy staging | develop, main | Chama o deploy hook do Render com imgURL apontando para a imagem exata e roda o smoke test: espera health UP e confere se /api/info já mostra o commit novo. |
| 4. Deploy produção | somente main | Só roda se staging passou. Promove a mesma imagem e repete o smoke test. O environment production pode exigir aprovação manual. |

## Lógica

- **Build único, promoção entre ambientes:** a imagem testada em staging é byte a byte a que vai para produção.
- **Falha interrompe o fluxo:** cada job depende do anterior (needs), então teste quebrado não gera imagem e staging com problema não chega a produção.
- **Segredos fora do código:** os deploy hooks ficam em secrets de cada GitHub Environment (staging e production); URLs em variáveis do environment.
- **Verificação real do deploy:** o smoke test só aprova quando a versão publicada é a do commit atual, evitando aprovar a versão antiga que ainda estava no ar.

# 3. Docker

## Arquitetura (docker-compose.yml)




- **Serviços:** app (API) e db (PostgreSQL 16 Alpine).
- **Volume:** pgdata preserva os dados do banco entre reinícios.
- **Rede:** esg-net (bridge); a API encontra o banco pelo nome db.
- **Variáveis:** lidas do .env (modelo em .env.example). Os arquivos env/staging.env.example e env/production.env.example simulam os dois ambientes lado a lado na mesma máquina.

## Imagem criada

| Item | Valor |
|---|---|
| Nome no registro | ghcr.io/mateusbhering/atividade-desafio-devops |
| Tags | sha-<commit> (imutável) e develop / main |
| Build | Multi-stage: maven:3.9-eclipse-temurin-21 → eclipse-temurin:21-jre-alpine |
| Execução | Usuário sem root (esg), porta 8080 ou $PORT, -XX:MaxRAMPercentage=75 |
| Health check | wget em /actuator/health a cada 30 s |

## Comandos usados

```
# construir e subir aplicação + banco
cp .env.example .env
docker compose up -d --build
docker compose ps

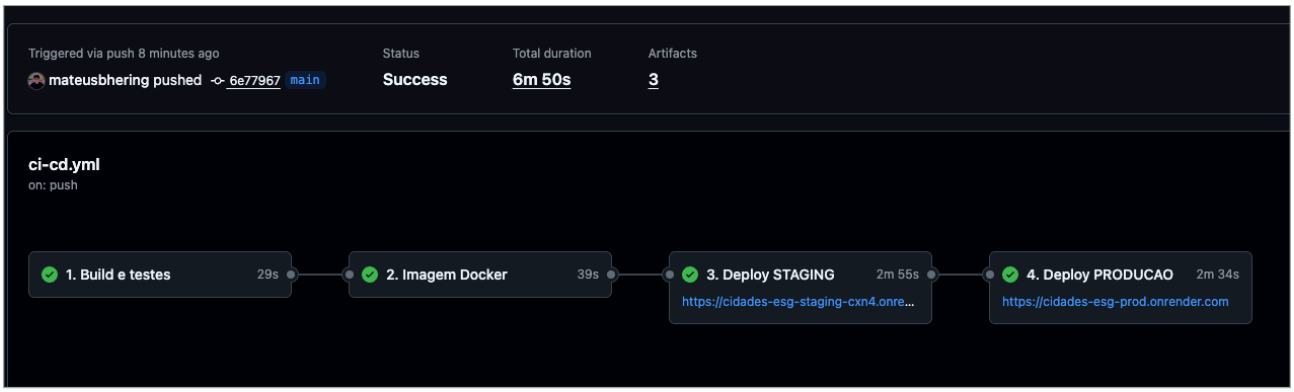
# testar
curl http://localhost:8080/api/info
bash scripts/smoke-test.sh http://localhost:8080

# staging e produção simulados localmente
docker compose -p esg-staging --env-file env/staging.env.example up -d --build
docker compose -p esg-prod --env-file env/production.env.example up -d --build

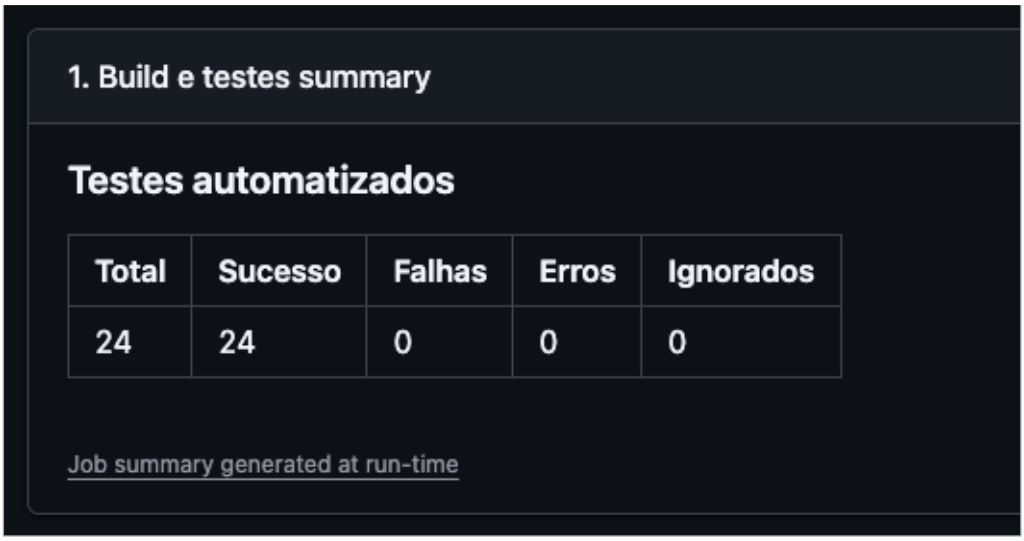
# imagem avulsa
docker build --build-arg APP_VERSION=local -t cidades-esg:local .

# derrubar (e apagar dados com -v)
docker compose down -v
```

# 4. Prints do pipeline




*Execução completa do pipeline no GitHub Actions (4 jobs concluídos)*




*Job 'Build e testes' com o resumo dos testes (24 testes, 0 falhas)*

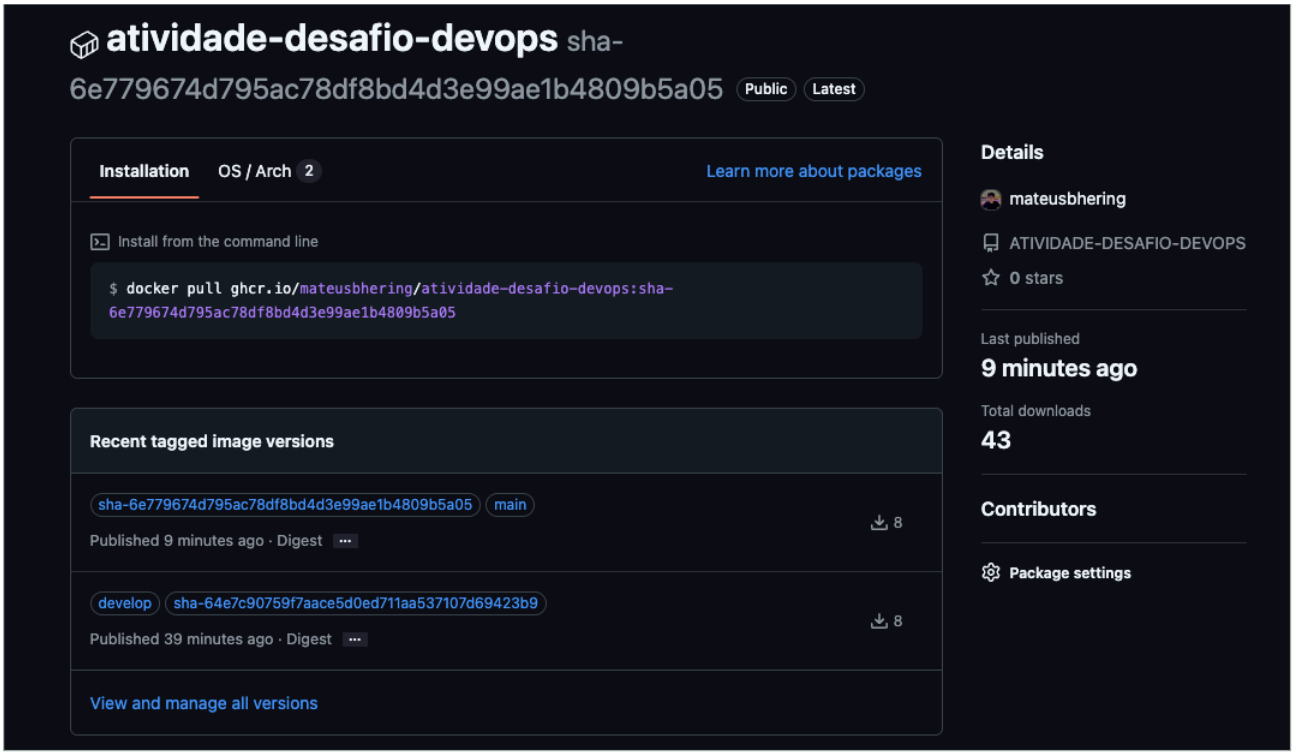



*Imagem publicada no GitHub Container Registry com a tag sha-<commit>*

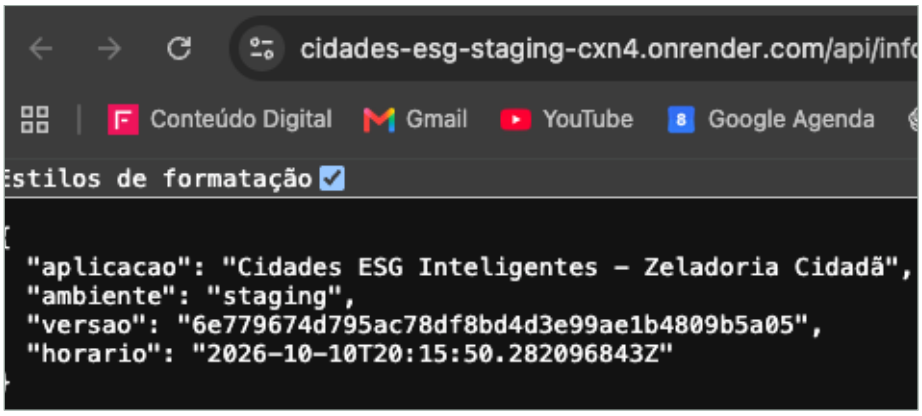



*Job 'Deploy STAGING' com o smoke test aprovado*

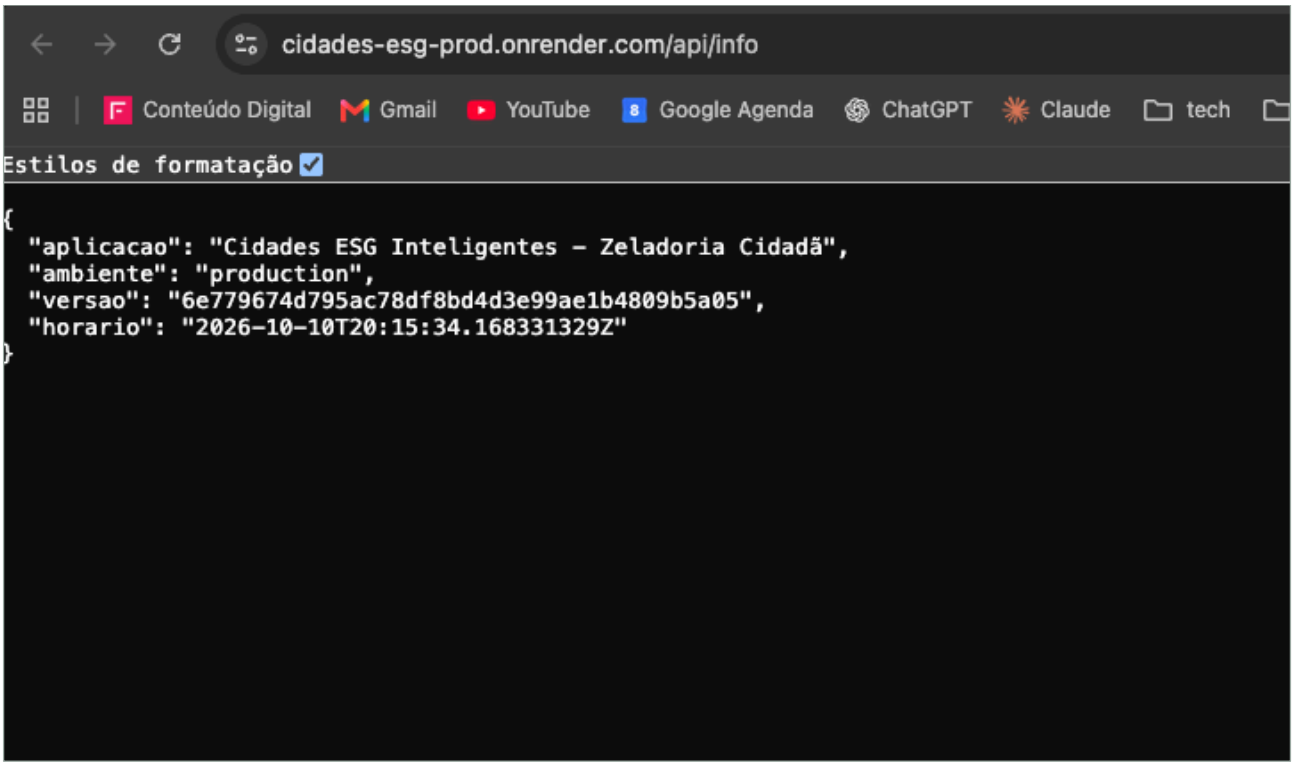



*Job 'Deploy PRODUCAO' com o smoke test aprovado*

# 5. Ambientes staging e produção funcionando




*Staging no ar: https://cidades-esg-staging-cxn4.onrender.com/api/info mostrando "ambiente":"staging"*




*Produção no ar: https://cidades-esg-prod.onrender.com/api/info mostrando "ambiente":"production"*




*Ambiente local: docker compose ps e a API respondendo em localhost:8080*




*Painel de transparência em staging: https://cidades-esg-staging-cxn4.onrender.com/api/transparencia com os dados de demonstração*

# 6. Desafios encontrados e soluções

| Desafio | Como foi resolvido |
|---|---|
| O plano gratuito do Render permite apenas um PostgreSQL por conta, mas havia dois ambientes. | Um único servidor com schemas separados (esg_staging e esg_prod), definidos pela variável DB_SCHEMA; o Hibernate cria o schema automaticamente. |
| Rodar os testes no pipeline sem depender de um banco externo. | Testes usam H2 em memória (modo PostgreSQL) via src/test/resources/application-test.yml (perfil test). |
| O Render não redeploya sozinho quando uma imagem nova chega ao registro. | O pipeline chama o deploy hook de cada serviço passando imgURL com a tag sha-<commit>. |
| O serviço gratuito do Render hiberna e leva cerca de 1 minuto para acordar; o deploy também não é instantâneo. | Smoke test com novas tentativas (a cada 10 s, por até 15 min) que só aprova quando /api/info mostra o commit novo. |
| Testar prazos e atrasos, que dependem da data e hora atuais, sem testes instáveis. | Todo cálculo de tempo usa um java.time.Clock injetado pelo Spring; nos testes, um Clock fixo torna o resultado sempre o mesmo. As regras ficam na entidade e são testadas sem banco. |
| O GHCR rejeita nomes de imagem com letras maiúsculas. | O nome é convertido para minúsculas no pipeline (${GITHUB_REPOSITORY,,}). |
| O primeiro serviço do Render foi criado a partir do repositório Git: o Render compilava a imagem sozinho, sem o commit gravado e com deploy a cada push, sem passar pelos testes. | Os serviços foram recriados como Existing Image apontando para o GHCR. Assim o pipeline é o único caminho de deploy e a mesma imagem é promovida de staging para produção. |
| O Render gera o nome do banco com sufixo aleatório e entrega a URL no formato postgres://usuario:senha@host/banco, que o driver JDBC não aceita. | A URL foi convertida para jdbc:postgresql://<host-interno>:5432/<banco>, com usuário e senha em SPRING_DATASOURCE_USERNAME e SPRING_DATASOURCE_PASSWORD. |
| O Render define a porta pela variável PORT e limita a memória em 512 MB. | server.port=${PORT:8080} e -XX:MaxRAMPercentage=75 no Dockerfile. |
| Não expor senhas no repositório. | .env no .gitignore e .dockerignore, .env.example como modelo e deploy hooks como secrets. |

# 7. Checklist de entrega

| Item | Status |
|---|---|
| Projeto compactado em .ZIP com estrutura organizada | Concluído |
| Dockerfile funcional | Concluído |
| docker-compose.yml ou arquivos Kubernetes | Concluído |
| Pipeline com etapas de build, teste e deploy | Concluído |
| README.md com instruções e prints | Concluído |
| Documentação técnica com evidências (PDF ou PPT) | Concluído |
| Deploy realizado nos ambientes staging e produção | Concluído |